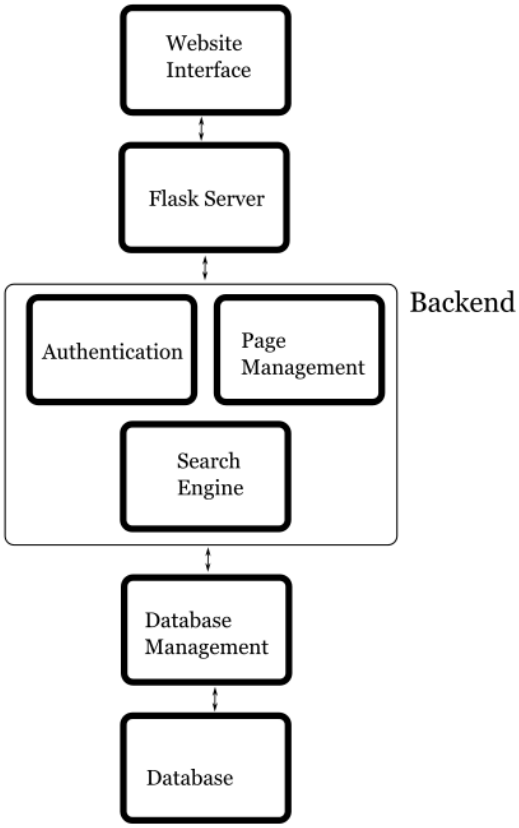
Design Doc - Hypergamy

TARGET SHIP DATE: 2026-11-17

Marcos Lehman, Leon Wong, Efrat Hossain, Wenjie Ma

Program Components

- **Flask server (serve.py) - Efrat** Central component of the website, connects the website's different components and displays appropriate pages.
- **Authentication server (auth.py) - Leon** Manages user registration, login, logout, sessions; ensures only authenticated users can edit the wiki.
- **Database management (data.py) - Efrat + Leon** Handles communication with the database; updates, retrieves, stores accounts, pages, and edit history
- **Page management (pages.py) - Wenjie + maybe other member -** Handles creating, viewing, editing wiki pages and records user contributions
- **Search engine (search.py) - Marcos + maybe other member -** Allows users to find pages via the searchbar; displays partial matches as search results
- **HTML templates (templates/) - made by anyone when it becomes relevant to their role** Contains HTML files for displaying home + login + profile + wiki pages; uses Flask templates to display info dynamically
- **Static files (static/) -** Stores CSS, controls website's appearance + layout



- The Flask server serves as the central point of communication between the user's browser (frontend) and the application's Python components (backend).
- When the user submits a request, Flask determines which components handles it (ex. login requests by authentication component, wiki editing requests by pages component)
- These components communicate with the database management component to retrieve / modify info; the database management component also handles SQLite queries and returns result to the application components → Flask renders the appropriate template and sends the result back to the browser.

ROUTES

`all routes (EXCEPT for /login and /new_user)`

At the top of every page, there will be a row of buttons. This makes doing certain things like logging in or out easier.

`Home` button -> Redirect to `/`

`Profile` button -> Redirect to `/profile`

`Make a page`

`if username in session:`

- `Logout` button -> Remove user from `session`. For ease of programming, always redirect the user to `/`.

`else:`

- `Login` button -> redirect to `/login?return-to=`(this route)
- `Sign Up` button -> redirect to `/new_user?return-to=`(this route)

`Search bar` (text field with a submit) -> Redirect to `/search?query=`(the thing the user searched for)

`/{pagename}/`

Redirect to the homepage if this page doesn't exist, also say thats why you got redirected (probably by using a querystring)

Else, just show the html that makes up the page.

`Edit` button (present to ALL visitors, logged in or not) -> redirect to `/{pagename}/edit`

`/{pagename}/edit`

`if username not in session:`

- Redirect to `/login?return-to=`(this route)

If the page does not exist, add it to the db with an empty contents.
but if it DOES exist, load the contents

https://www.mediawiki.org/wiki/Help:Edit_conflict as a reference for the following

`Save` button -> saves your changes to the contents and tries to put it in the db. Reload this page, and you will see different things as detailed by the following.

- if it merges cleanly, then yippee! Say that successfully saved.
- if there is a merge conflict, then load this page in a secondary mode: merge conflict mode.
    - shows a different template displaying the existing version and the version you tried to make side by side, with differences highlighted somehow. Then in a third box below is the actual wiki page, you can decide for yourself what to include or not. Same saving thing here.

`/`

Root route. This is the home page. Has info on it, idk what. Probably an extra button for logging in even though there's already one at the top for every page. (this will make the page more inviting, so still prob good to have)

`login` button -> redirect to `/login?return-to=/`

`/search? (query=QUERY)`

`if 'query' not in request.args:`

- redirect to `/`.

If query matches perfectly with a listed page in the dataset and there is only one match, redirect there.
Else, do some fuzzy search thing and show potential candidates for what the user wanted.

- idk if sql has a fuzzy searching thing, we might need to not use a fuzzy search and just do something like `SELECT name FROM pages WHERE name LIKE "%query%"`

/login? (return-to=PATH)

`if` logged in, redirect to `[return-to]`or to `/`

Prompts you to log in with an existing account.

`submit` button -> check if username and password match?

- true: add user to `session`. Redirect to `[return-to]`or to `/`
- false: redirect to the current page, same query string and everything. Tell the user they done messed up.

`"Don't have an account?"` button -> redirect to `/new_user?return-to=[return-to]`

/new_user? (return-to=PATH)

`if` logged in, redirect to `[return-to]`or to `/`

Asks the user to make a new account. Provide fields for username and password.

submit button ->

- if username is taken (username exists in the db), refresh and tell the user that.
- else, register the user with the password (add it to db with some encrypting on it prob), log them in (add to `session`), and redirect them to `[return-to]`or to `/`

/profile

`if username not in session:`

- Redirect to `/login?return-to=/profile`.

Displays the users profile, which for now is just the list of pages with contributions.

Each one is a link that takes you to the corresponding page.

First section lists ones you created.
Second section lists ones you didn't create, but you edited them.

DATABASES

credentials

`(username TEXT, password BLOB, uuid INTEGER PRIMARY KEY)`

Stores info about a specific user.

- username is their username.
- password is some encrypted form of their password.
- uuid is their unique identifier.

pages

`(title TEXT, contents BLOB, creator INTEGER, contributors TEXT, puid INTEGER PRIMARY KEY)`

Stores info about a specific page.

- title is the title of the page, stored separately for ease of querying.
- contents is the html contents of the page. stored as a blob so no weird things happen when saving.
- creator is the one person who created it, it's their uuid.
- contributors is a string of uuid's.
    - the way this works is its stored like this: ",354,63,1,559,"
    - so if you want to find which pages that a specific user edited, you can query like so: SELECT title IN pages WHERE contributors LIKE "%,uuid,%"

Frontend Map

